

Corso di Programmazione Web
Simulazione di esame – 28 Maggio 2025

- Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.
- Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.
- Fornire esempi di richieste e risposte per mostrare il funzionamento.
- Non scrivere codice effettivo, ma fornire dettagliate progettazioni API, i formati di richiesta/risposta e i modelli di dati.

Progettare un sistema web che gestisca le prenotazioni in un ristorante. Il sistema deve prevedere:

- Un'API per registrare un nuovo ristorante (nome, indirizzo, numero massimo di coperti);
- Un'API per ottenere la lista di tutti i ristoranti disponibili;
- Un'API per visualizzare le informazioni di un ristorante dato il suo ID;
- Un'API per prenotare un tavolo indicando nome, numero di persone e orario.

Domanda 1: Definizione delle API (15 punti)

- Definire i modelli di dati per richiesta e risposta. Inserire i tipi di dato per ogni campo ed eventuali constraint – **3 punti in totale**
- Definire endpoint API (URL + metodo HTTP) e descriverlo testualmente dove necessario – **2 punti per ciascuna API**
- Specificare i parametri di richiesta e le risposte previste, fornendo un esempio di richiesta con successo per ognuna – **1 punto per ciascuna API**

Domanda 2: Gestione degli Errori (5 punti)

- Descrivere i codici di errore rilevanti (400, 404, 500) – **1 punto**
- Fornire almeno due esempi concreti di risposta di errore – **2 punti ciascuno (max 4 punti in totale)**

Alcune domande possibili da 3 punti:

1. Supponiamo che il sistema venga usato contemporaneamente da migliaia di utenti durante le ore di punta (es. cena del sabato). Indica almeno **due strategie tecniche** che potresti applicare per migliorare la **scalabilità e le performance** del sistema. Spiega in breve il ruolo di ciascuna.

2. Descrivi **due buone pratiche per il deployment automatico** del sistema di prenotazioni ristoranti, in modo che possa essere facilmente aggiornato e mantenga l'affidabilità in produzione.
3. Il sistema attuale permette a chiunque di fare prenotazioni o registrare ristoranti. Spiega **come introdurre autenticazione** per gli utenti e **come gestire ruoli diversi** (es. utente normale vs. gestore ristorante).
4. Dopo che un utente effettua una prenotazione, il sistema deve inviargli un'email di conferma. Spiega **perché è meglio gestire l'invio in modo asincrono**, e **quali strumenti potresti usare per implementarlo**.
5. Per garantire che le API funzionino correttamente, è importante testarle. Descrivi almeno **due tipi di test automatici** che scriveresti per questo progetto e **quale parte del sistema andrebbero a verificare**.

Esempio di soluzione attesa (parziale)

Modello Ristorante

```
{
  "id": int,
  "name": str,
  "address": str,
  "maxGuests": int, min=0, max=200
}
```

Modello Prenotazione

```
{
  "name": str,
  "people": int, min=1, max=10
  "time": datetime | str
  "restaurant": int -> id del ristorante
}
```

1. Inserimento nuovo ristorante

POST /restaurants

Richiesta:

```
{
  "name": "Trattoria Bella Napoli",
  "address": "Via Roma 10, Napoli",
  "maxGuests": 50
}
```

Risposta alternativa

Utilizzo del modello “Restaurant” senza specificare l’ID.

Risposta:

```
{
  "id": 1,
  "message": "Ristorante registrato con successo"
}
```

Altra risposta possibile:

202, created

2. Lista ristoranti

GET /restaurants

Risposta:

```
[
  {
    "id": 1,
```

```
[
  {
    "name": "Trattoria Bella Napoli",
    "address": "Via Roma 10, Napoli",
    "maxGuests": 50
  }
]
```

3. Dettagli ristorante

GET /restaurants/{id}, id è un parametro di path

Esempio

GET /restaurants/1

Risposta:

Il server restituirà un oggetto "Ristorante", esempio:

```
{
  "id": 1,
  "name": "Trattoria Bella Napoli",
  "address": "Via Roma 10, Napoli",
  "maxGuests": 50
}
```

4. Prenotazione tavolo

POST /restaurants/{id}/reservations

Esempio:

POST /restaurants/1/reservations

Richiesta:

```
{
  "name": "Luca Bianchi",
  "people": 4,
  "time": "2025-06-01T20:00"
}
```

Risposta:

200 OK

Gestione degli Errori

- **400 Bad Request:** Parametri errati o mancanti (es. numero persone non valido)
- **404 Not Found:** ID ristorante inesistente
- **500 Internal Server Error:** Errore nella scrittura dati sul server

Esempio errore 400/422:

Cause comuni di questo errore possono essere

- Il numero di persone non è un numero intero positivo.

- Il campo "name" è mancante nella prenotazione.
- Il formato dell'orario è errato.

Richiesta:

```
{
  "name": "Luca Bianchi",
  "people": -10,
  "time": "2025-06-01T20:00"
}
```

```
{
  "error": "Numero di persone non valido"
}
```

Esempio errore 404:

Questo errore può essere causato da una GET per un ristorante non esistente.

Per esempio,

```
GET /restaurants/42
```

Risposta:

```
{
  "error": "Ristorante con ID 42 non trovato"
}
```

Esempio errore 500:

Un errore imprevisto lato server, ad esempio:

- Il database è temporaneamente non disponibile.
- Errore di sistema nella scrittura dei dati.

```
{
  "error": "Errore interno del server. Riprova più tardi."
}
```

Risposte alle domande da 3 punti:

1. Supponiamo che il sistema venga usato contemporaneamente da migliaia di utenti durante le ore di punta (es. cena del sabato). Indicare almeno **due strategie tecniche** che si potrebbero applicare per migliorare la **scalabilità e le performance** del sistema. Spiegare in breve il ruolo di ciascuna.
 - **Possibile risposta.** Il sistema può essere adattato per migliorare la scalabilità implementando una coda di processi che gestisca le richieste degli utenti in modo asincrono rispetto al processing. Il server girerà su un processo dedicato e metterà in coda le operazioni di processing lungo. Un processo separato, eventualmente su una macchina diversa, preleverà i task dalla coda e effettuerà le operazioni più lente. In questo modo, il server potrà continuare a ricevere richieste senza interrompere il funzionamento. Un altro metodo per aumentare la scalabilità è quello di fornire al server più risorse, per esempio tramite l'upgrade dell'hardware,

o l'acquisto di un servizio di livello più alto (per esempio più RAM, una CPU più veloce) nel caso di deployment su piattaforme di internet-as-a-service.

2. Descrivere **due buone pratiche per il deployment automatico** del sistema di prenotazioni ristoranti, in modo che possa essere facilmente aggiornato e mantenga l'affidabilità in produzione.
 - **Possibile Risposta.** Per gestire l'aggiornamento rapido e facilitare la gestione del sistema, si può prevedere l'utilizzo di container per velocizzare la configurazione e l'installazione di nuove funzionalità. Inoltre, questo servizio può essere connesso a tecnologie di Continuous Integration and Delivery / Deployment per eseguire un workflow automatico che si attiva periodicamente o come conseguenza di azioni (per esempio un push sul main branch di un repository).
3. Il sistema attuale permette a chiunque di fare prenotazioni o registrare ristoranti. Spiega **come introdurre autenticazione** per gli utenti e **come gestire ruoli diversi** (es. utente normale vs. gestore ristorante).
 - **Possibile Risposta.** Il sistema può prevedere un metodo di autenticazione che identifica l'utente specifico e abilita l'accesso alle risorse che l'utente può leggere (permesso di lettura) o modificare (permesso di scrittura). Gli utenti possono essere divisi in diversi ruoli per effettuare una role-based authorization, a seconda dei privilegi che si vogliono concedere. Si prevedono tre ruoli: uno di admin, accessibile solo al gestore di sistema e con i massimi privilegi; uno di gestore ristorante, che può creare un ristorante, leggere le informazioni dei ristoranti, modificare quelle del proprio ristorante, e vedere o modificare prenotazioni; e uno di utente, che permette solo di visualizzare i ristoranti ed effettuare, visualizzare o modificare le proprie prenotazioni. Gli utenti non possono vedere o modificare le prenotazioni degli altri utenti, e i gestori di ristoranti non possono vedere o modificare le prenotazioni degli altri ristoranti, e non possono modificare le informazioni degli altri ristoranti.
4. Dopo che un utente effettua una prenotazione, il sistema deve inviargli un'email di conferma. Spiega **perché è meglio gestire l'invio in modo asincrono**, e **quali strumenti potresti usare per implementarlo**.
 - **Possibile Risposta.** Il sistema dovrebbe inviare le mail in modo asincrono per consentire di disaccoppiare la ricezione della richiesta dall'operazione di invio mail. In particolare, se il server di mail risulta lento o non raggiungibile, il server potrebbe rimanere bloccato per cercare di completare comunque l'operazione. L'utente non solo non riceverebbe conferma immediata, ma potrebbe anche erroneamente reinviare la richiesta tramite refresh della pagina, causando ulteriori rallentamenti. Inoltre, il server rimarrebbe bloccato anche per altri utenti che stanno cercando di inviare le loro richieste. Una possibile soluzione a questo problema è l'implementazione di una coda di processi che archivia le

richieste di prenotazione degli utenti e invia le mail in modo asincrono (vedere anche risposta a domanda 1 sopra, nella parte relativa alla coda).

5. Per garantire che le API funzionino correttamente, è importante testarle.

Descrivi almeno **due tipi di test automatici** che scriveresti per questo progetto e **quale parte del sistema andrebbero a verificare**.

- **Possibile Risposta.** Un possibile test per le API appena progettate potrebbe essere la verifica che una richiesta corretta vada a buon fine. A tal proposito, si può eseguire una chiamata HTTP e ottenere la risposta dal server, verificando che la risposta sia quella attesa in fase di progettazione (per esempio, verificare che restituisca il corretto status code, o il corretto messaggio di risposta). Un altro possibile test potrebbe verificare che una API restituisca un errore in caso di richiesta malformata. Per esempio, simulando un utente che indichi un numero non intero nel numero di persone, e verificando che il server risponda con lo status code corretto e che non si blocchi.